Spring Boot — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction to Spring and Spring Boot

02 Project Structure and First Application

03 Inversion of Control and Dependency Injection

04 Beans and Stereotype Annotations

05 REST Controllers

06 Configuration and Profiles

Intermediate (6 topics)

07 Spring Data JPA

08 Entity Relationships

09 Validation and DTOs

10 Exception Handling

11 Database Migrations

12 Testing

Advanced (6 topics)

13 Spring Security

14 JWT and OAuth2

15 AOP and Caching

16 Actuator and Observability

17 Messaging: Kafka and RabbitMQ

18 Scheduling and Async

Master (4 topics)

19	Microservices with Spring Cloud
20	Docker, Kubernetes and CI/CD
21	Performance and Production Readiness
22	Real Project: E-commerce / Banking API

Beginner Level

01. Introduction to Spring and Spring Boot

DEFINITION

Spring is a Java framework built around dependency injection. **Spring Boot** adds auto-configuration, starters and an embedded server so you can run an app with `java -jar`.

SYNTAX

```
start.spring.io
```

EXAMPLE

```
Generate a project at start.spring.io (Java 21, Maven, Spring Web)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Buying a ready-to-use kitchen set instead of making every utensil; Spring Boot gives a configured app and server in minutes.

தமிழ்

ஒவ்வொரு பாத்திரத்தையும் செய்யாமல் தயாரான சமையல் தொகுப்பு வாங்குவது போல Spring Boot நிமிடங்களில் தயாரான செயலியையும் server-ஐயும் தருகிறது.

PROS

- Auto-configuration
- Embedded server
- Production-ready features
- Huge ecosystem

CONS

- Heavy memory and startup
- Magic can hide what happens
- Large dependency tree

ADVANCED / NEW FEATURES

- Spring Boot 4 on Spring Framework 7
- Virtual threads support
- GraalVM native images
- Java 17+ baseline

02. Project Structure and First Application

DEFINITION

`@SpringBootApplication` marks the entry point; `application.properties` configures the app.

SYNTAX

```
@SpringBootApplication
```

EXAMPLE

```
@SpringBootApplication
public class App {
    public static void main(String[] args) { SpringApplication.run(App.class, args); }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

The master switch of a building: pressing one switch powers the whole house; `@SpringBootApplication` starts configuration and scanning.

தமிழ்

கட்டிடத்தின் முதன்மை சுவிட்ச் போல `@SpringBootApplication` configuration, scanning அனைத்தையும் தொடங்குகிறது.

PROS

- Zero XML
- Convention over configuration

CONS

- Hidden defaults

ADVANCED / NEW FEATURES

- Faster startup with AOT
- Docker Compose support (3.1+)

03. Inversion of Control and Dependency Injection

DEFINITION

The container creates beans and injects dependencies. Prefer constructor injection.

SYNTAX

```
@Service class A { A(B b) { } }
```

EXAMPLE

```
@Service
class OrderService {
```

```
    private final PaymentService payments;
    OrderService(PaymentService payments) { this.payments = payments; }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shopkeeper does not make soap himself; the supplier (container) delivers it. Dependency injection supplies required objects.

தமிழ்

கடைக்காரர் சோப்பைத் தானே தயாரிக்க மாட்டார்; சப்ளையர் (container) கொண்டு வந்து கொடுக்கிறார். Dependency injection தேவையான objects-ஐ வழங்கும்.

PROS

- Loose coupling
- Easy testing and mocking

CONS

- Circular dependencies
- Reflection overhead

ADVANCED / NEW FEATURES

- Constructor injection recommended
- `@Autowired` often optional

04. Beans and Stereotype Annotations

DEFINITION

`@Component`, `@Service`, `@Repository`, `@Controller`, `@Configuration` + `@Bean` register beans.

SYNTAX

```
@Component / @Service / @Repository / @Bean
```

EXAMPLE

```
@Configuration class Config { @Bean Clock clock() { return Clock.systemUTC(); } }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Each staff member has a role badge (manager, cashier, storekeeper); stereotype annotations label beans' roles.

தமிழ்

ஒவ்வொரு பணியாளருக்கும் பங்கு அடையாள அட்டை (மேலாளர், காசாளர், கிடங்கு) போல stereotype annotations beans-இன் பங்கைக் குறிக்கும்.

PROS

- Clear architecture
- Auto detection

CONS

- Annotation overuse hides wiring

ADVANCED / NEW FEATURES

- `@ConfigurationProperties` records

05. REST Controllers

DEFINITION

`@RestController`, `@GetMapping`, `@PostMapping`, `@PathVariable`, `@RequestParam`, `@RequestBody`.

SYNTAX

```
@GetMapping("/x")
@PostMapping
```

EXAMPLE

```
@RestController @RequestMapping("/api/users")
class UserController {
    @GetMapping("/{id}") User get(@PathVariable Long id) { return service.find(id); }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A reception that routes visitors by purpose; controller methods map URLs to code.

தமிழ்

நோக்கத்தைப் பொறுத்து பார்வையாளர்களை அனுப்பும் வரவேற்பு போல controller methods URL-ஐ குறியீட்டுடன் இணைக்கின்றன.

PROS

- Concise REST APIs
- JSON handling built in

CONS

- Fat controllers if misused

ADVANCED / NEW FEATURES

- `RestClient` (3.2)

- HTTP interface clients
- ProblemDetail errors

06. Configuration and Profiles

DEFINITION

Use `application.yml`, `@Value`, `@ConfigurationProperties` and profiles (`dev`, `prod`).

SYNTAX

```
application.yml
@Value("${x}")
```

EXAMPLE

```
server.port=8081
spring.profiles.active=dev
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop with different price lists for festival and normal days; profiles switch configuration per environment.

தமிழ்

பண்டிகை, சாதாரண நாட்களுக்கு வெவ்வேறு விலைப்பட்டியல் போல profiles சூழலுக்கேற்ப configuration-ஐ மாற்றுகின்றன.

PROS

- Environment-specific config
- Type-safe properties

CONS

- Misconfigured profiles in production

ADVANCED / NEW FEATURES

- Config import
- Docker Compose integration (3.1+)

Intermediate Level

07. Spring Data JPA

DEFINITION

Map entities with `@Entity` and use repositories for CRUD without SQL.

SYNTAX

```
interface R extends JpaRepository<E, Long> { }
```

EXAMPLE

```
@Entity class User { @Id @GeneratedValue Long id; String name; }
interface UserRepo extends JpaRepository<User, Long> { List<User> findByName(String name); }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A clerk who writes entries in the ledger for you; Spring Data creates queries from method names.

தமிழ்

உங்களுக்காக பதிவேட்டில் எழுதும் எழுத்தர் போல Spring Data method பெயர்களிலிருந்து queries உருவாக்கும்.

PROS

- Almost no SQL for CRUD
- Pagination and sorting built in

CONS

- N+1 queries
- Abstraction hides SQL

ADVANCED / NEW FEATURES

- Spring Data JDBC
- Hibernate 7 (Boot 4)

08. Entity Relationships

DEFINITION

`@OneToMany`, `@ManyToOne`, `@ManyToMany`; lazy vs eager loading; avoid N+1 queries with fetch joins.

SYNTAX

```
@OneToMany(mappedBy = "x")
```

EXAMPLE

```
@OneToMany(mappedBy = "author") List<Post> posts;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A customer has many orders; each order belongs to one customer. Relationships map such links to tables.

தமிழ்

ஒரு வாடிக்கையாளருக்கு பல ஆர்டர்கள்; ஒவ்வொரு ஆர்டரும் ஒரு வாடிக்கையாளருக்கு; relationships அவற்றை அட்டவணைகளாக இணைக்கும்.

PROS

- Object model matches domain

CONS

- Lazy loading exceptions
- N+1 problem

ADVANCED / NEW FEATURES

- Entity graphs
- Hibernate batch fetching

09. Validation and DTOs

DEFINITION

Validate input with Bean Validation (`@NotBlank`, `@Email`, `@Valid`); expose DTOs/records, not entities.

SYNTAX

```
record R(@NotBlank String name) { }
@Valid @RequestBody R r
```

EXAMPLE

```
record CreateUser(@NotBlank String name, @Email String email) {}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A form with mandatory fields checked at the counter; DTOs and Bean Validation check the data before use.

தமிழ்

கவுண்டரில் கட்டாயப் புலங்கள் சரிபார்க்கப்படும் படிவம் போல DTO, Bean Validation தரவைப் பயன்படுத்தும் முன் சரிபார்க்கின்றன.

PROS

- Clean API boundaries
- Declarative validation

CONS

- Duplicate DTO classes

ADVANCED / NEW FEATURES

- Records as DTOs
- Jakarta Validation 3

10. Exception Handling

DEFINITION

Centralise errors with `@RestControllerAdvice` and `@ExceptionHandler`; use ProblemDetail.

SYNTAX

```
@RestControllerAdvice
```

EXAMPLE

```
@ExceptionHandler(NotFound.class)
ResponseEntity<String> nf(NotFound e) { return
ResponseEntity.status(404).body(e.getMessage()); }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A single complaint counter handling all customer problems with consistent replies.

தமிழ்

அனைத்து வாடிக்கையாளர் குறைகளையும் ஒரே மாதிரி பதிலுடன் கையாளும் ஒரே புகார் கவுண்டர்.

PROS

- Consistent error responses
- Less try/catch

CONS

- Over-generic handlers hide causes

ADVANCED / NEW FEATURES

- `ProblemDetail` (RFC 9457) support

11. Database Migrations

DEFINITION

Manage schema with Flyway or Liquibase instead of `ddl-auto=update`.

SYNTAX

```
V1__init.sql
```

EXAMPLE

```
src/main/resources/db/migration/V1__create_users.sql
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A building's renovation log: every change recorded with version; Flyway tracks database changes.

தமிழ்

கட்டிட புதுப்பிப்புப் பதிவேடு போல ஒவ்வொரு மாற்றமும் பதிவாகும்; Flyway database மாற்றங்களைப் பதிவு செய்கிறது.

PROS

- Repeatable schema changes
- Team-safe

CONS

- Needs discipline

ADVANCED / NEW FEATURES

- Flyway/Liquibase auto-config
- Testcontainers integration

12. Testing

DEFINITION

`@SpringBootTest`, `@WebMvcTest`, MockMvc, Mockito and Testcontainers.

SYNTAX

```
@SpringBootTest
@WebMvcTest
```

EXAMPLE

```
mockMvc.perform(get("/api/users/1")).andExpect(status().isOk());
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A trial run of the kitchen before the restaurant opens; tests check the app at different layers.

தமிழ்

உணவகம் திறக்கும் முன் சமையலறை சோதனை ஓட்டம் போல tests அடுக்குகளைச் சோதிக்கின்றன.

PROS

- Slice tests are fast
- Real DB via Testcontainers

CONS

- Full-context tests are slow

ADVANCED / NEW FEATURES

- `@ServiceConnection` (3.1)
- `RestTestClient` (Boot 4)

Advanced Level

13. Spring Security

DEFINITION

Authentication and authorisation with filters, `SecurityFilterChain`, password encoders, method security.

SYNTAX

```
http.authorizeHttpRequests(...)
```

EXAMPLE

```
@Bean SecurityFilterChain chain(HttpSecurity http) throws Exception {
    return http.authorizeHttpRequests(a ->
a.requestMatchers("/public/**").permitAll().anyRequest().authenticated()).build();
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Security guards, ID checks and special passes for sensitive rooms; Spring Security handles login and permissions.

தமிழ்

பாதுகாப்புக் காவலர்கள், அடையாளச் சோதனை, முக்கிய அறைகளுக்கான சிறப்பு அனுமதி போல Spring Security login, permissions-ஐக் கையாளும்.

PROS

- Battle-tested
- Flexible

CONS

- Steep learning curve
- Config mistakes are risky

ADVANCED / NEW FEATURES

- Lambda DSL (no `WebSecurityConfigurerAdapter`)
- Passkeys support
- OAuth 2.1 trends

14. JWT and OAuth2

DEFINITION

Stateless APIs with JWT, resource server and login with Google/GitHub via OAuth2.

SYNTAX

```
issuer-uri: https://...
```

EXAMPLE

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://issuer
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bus pass issued by a trusted authority; tokens from an identity provider prove who you are.

தமிழ்

நம்பகமான அதிகாரியால் வழங்கப்பட்ட பேருந்து பாஸ் போல identity provider token நீங்கள் யார் என நிரூபிக்கிறது.

PROS

- No password handling
- Standard SSO

CONS

- Token validation setup

ADVANCED / NEW FEATURES

- OAuth2 resource server
- Authorization server project

15. AOP and Caching

DEFINITION

Cross-cutting concerns with `@Aspect`; caching with `@Cacheable` and Redis.

SYNTAX

```
@Cacheable("x")
```

EXAMPLE

```
@Cacheable("users") public User find(Long id) { /* ... */ }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A tea master keeps frequently ordered cups ready; caching stores results to avoid repeated work.

தமிழ்

அடிக்கடி கேட்கப்படும் கோப்பைகளை தேநீர் மாஸ்டர் தயாராக வைத்திருப்பது போல caching முடிவுகளைச் சேமிக்கிறது.

PROS

- Big speed-ups
- Declarative

CONS

- Stale data
- Cache invalidation is hard

ADVANCED / NEW FEATURES

- Redis/Caffeine support
- Cache metrics in Actuator

16. Actuator and Observability

DEFINITION

Health, metrics and info endpoints, Micrometer, Prometheus, tracing.

SYNTAX

```
management.endpoints.web.exposure.include=health
```

EXAMPLE

```
management.endpoints.web.exposure.include=health,metrics,prometheus
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A dashboard in the cab showing fuel, speed and temperature; Actuator shows the app's health and metrics.

தமிழ்

வாகன டாஷ்போர்டில் எரிபொருள், வேகம், வெப்பநிலை போல Actuator செயலியின் ஆரோக்கியம், metrics-ஐ காட்டுகிறது.

PROS

- Production readiness

- Micrometer metrics

CONS

- Exposed endpoints must be secured

ADVANCED / NEW FEATURES

- OpenTelemetry integration
- Observation API

17. Messaging: Kafka and RabbitMQ

DEFINITION

Event-driven communication with Spring Kafka/AMQP; producers, consumers and retries.

SYNTAX

```
@KafkaListener(topics = "t")
```

EXAMPLE

```
@KafkaListener(topics = "orders") void on(String msg) { }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A post office sorting parcels into pigeonholes that different teams collect from; message brokers decouple services.

தமிழ்

பார்சல்களை பெட்டிகளில் தரம் பிரித்து குழுக்கள் எடுத்துச் செல்வது போல message brokers சேவைகளைப் பிரிக்கின்றன.

PROS

- Loose coupling
- High throughput

CONS

- Operational complexity
- Eventual consistency

ADVANCED / NEW FEATURES

- Spring Kafka retry topics
- Spring Cloud Stream

18. Scheduling and Async

DEFINITION

`@Scheduled`, `@Async`, virtual threads support.

SYNTAX

```
@Scheduled(cron = "0 0 * * * *")
@Async
```

EXAMPLE

```
@Scheduled(cron = "0 0 * * * *") void hourly() { }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

An alarm clock that wakes you daily; scheduled jobs and async tasks do background work.

தமிழ்

தினமும் எழுப்பும் அலாரம் போல scheduled jobs, async tasks பின்புற வேலையைச் செய்கின்றன.

PROS

- Simple automation
- Non-blocking tasks

CONS

- Duplicate runs in multiple instances
- Hard to monitor

ADVANCED / NEW FEATURES

- Virtual-thread executors
- Distributed schedulers (ShedLock, Quartz)

Master Level

19. Microservices with Spring Cloud

DEFINITION

Service discovery, API gateway, config server, resilience (circuit breakers), distributed tracing.

EXAMPLE

```
Spring Cloud Gateway + Eureka + Resilience4j
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A group of small shops cooperating through a main gate instead of one giant mall; microservices split an app into independent services.

தமிழ்

ஒரு பெரிய வணிக வளாகத்துக்குப் பதிலாக பிரதான வாயில் வழியாக ஒத்துழைக்கும் சிறு கடைகள் போல microservices செயலியை சுயாதீன சேவைகளாகப் பிரிக்கின்றன.

PROS

- Independent scaling and deployment
- Team autonomy

CONS

- Distributed system complexity
- Network failures
- Hard debugging

ADVANCED / NEW FEATURES

- Spring Cloud Gateway
- Kubernetes-native service discovery
- Resilience4j

20. Docker, Kubernetes and CI/CD

DEFINITION

Build images with buildpacks (`spring-boot:build-image`), deploy to Kubernetes, automate with GitHub Actions/GitLab CI.

SYNTAX

```
./mvnw spring-boot:build-image
```

EXAMPLE

```
./mvnw spring-boot:build-image
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Packing food in standard tiffin carriers for delivery; container images run anywhere.

தமிழ்

உணவை தரப்படுத்தப்பட்ட டிபன் கேரியர்களில் அனுப்புவது போல container images எங்கும் இயங்கும்.

PROS

- Consistent environments
- Cloud-ready

CONS

- Image size and patching

ADVANCED / NEW FEATURES

- Buildpacks
- Layered jars
- GraalVM native images

21. Performance and Production Readiness

DEFINITION

Connection pools, caching, pagination, indexes, GraalVM native images, graceful shutdown.

SYNTAX

```
server.shutdown=graceful
```

EXAMPLE

```
server.shutdown=graceful
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop that finishes serving the queue before closing the shutter; graceful shutdown and tuning keep production smooth.

தமிழ்

வரிசையில் உள்ளவர்களுக்கு சேவை செய்த பிறகு கதவை மூடுவது போல graceful shutdown, tuning உற்பத்தியை சீராக வைக்கின்றன.

PROS

- Fewer errors on deploy
- Better throughput

CONS

- Needs measurement

ADVANCED / NEW FEATURES

- Virtual threads
- CRaC checkpoint/restore
- AOT processing

22. Real Project: E-commerce / Banking API

DEFINITION

Layered REST API: JPA + PostgreSQL, security with JWT, validation, tests, Docker, CI/CD.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A complete online shop backend: users, products, orders, payments.

தமிழ்

பயனர்கள், பொருட்கள், ஆர்டர்கள், கட்டணங்கள் அடங்கிய முழு ஆன்லைன் கடை backend.

PROS

- Great portfolio project

CONS

- Takes time

ADVANCED / NEW FEATURES

- Use Boot 4 + Java 25 + Testcontainers

Interview Question Bank

Q1. What is Spring Boot?
A framework on top of Spring providing auto-configuration, starters and an embedded server for production-ready apps quickly.

Q2. What does `@SpringBootApplication` do?
Combines `@Configuration`, `@EnableAutoConfiguration` and `@ComponentScan`.

Q3. What is dependency injection?
The container supplies a class's dependencies; constructor injection is recommended.

Q4. `@Component` vs `@Service` vs `@Repository`?
All register beans; the latter two add meaning, and `@Repository` also translates persistence exceptions.

Q5. What is auto-configuration?
Spring Boot configures beans based on classpath libraries and properties.

Q6. `@RestController` vs `@Controller`?
`@RestController` = `@Controller` + `@ResponseBody` (returns data, not views).

Q7. How do you avoid the N+1 problem?
Use fetch joins, entity graphs or batch fetching.

Q8. What is a bean scope?
Lifecycle of a bean: singleton (default), prototype, request, session.

Q9. How do you secure a REST API?
Spring Security with JWT/OAuth2, HTTPS, validation, role-based access.

Q10. What are Actuator endpoints?
Production endpoints exposing health, metrics and runtime information.

Capstone Project Ideas

1. Employee management REST API
2. Blog with JWT auth
3. Banking microservices
4. E-commerce order service with Kafka
5. URL shortener with Redis